

Trabajo práctico Nro. 1

Programación funcional y orientada a objetos

Nombre del grupo:

Integrantes:

Líder:

Fecha:

Criterios de evaluación:

- Cumplir con todos los requisitos especificados en el enunciado del problema.
- Código fácil de leer y entender, con nombres de variables y funciones descriptivos, comentarios útiles y una indentación adecuada.
- Código eficiente en términos de tiempo y espacio, evitando cálculos innecesarios y el uso de estructuras de datos ineficientes.
- Uso óptimo de las funciones lambda, sort y filter.
- Las clases deben representar objetos del mundo real de manera adecuada, con atributos y métodos bien definidos.

Ejercicios:

1. Dada una lista de direcciones de correo electrónico, utiliza map() y una función lambda para extraer el dominio de cada correo (la parte después del "@").
2. Dada una lista de nombres y apellidos, utiliza map() y una función lambda para generar un código de usuario para cada persona, por ejemplo, las tres primeras letras del nombre seguidas de las tres primeras letras del apellido.
3. Dada una lista de libros representada por un diccionario, utiliza filter() y una función lambda para seleccionar solo los objetos libros con más de 300 páginas. Los atributos del libro son: nombre, autor, páginas.
4. Dada una lista que puede contener duplicados, utiliza filter() y una función lambda para obtener una nueva lista que contenga solo los elementos únicos (puedes combinarlo con otras estructuras de datos si es necesario).
5. Dada una lista de fechas, utiliza sorted() y una función lambda para ordenarlas cronológicamente.
6. Dada una lista de cadenas, utiliza sorted() y una función lambda para ordenarlas según la frecuencia con la que aparece una letra específica en cada cadena.
7. Sistema de Reservas de Cine: Modelar un sistema de reservas de cine con clases como Película, Sala y Reserva. Implementar métodos para mostrar películas, buscar salas disponibles y realizar reservas.
8. Empleado: Crea un sistema para gestionar los datos de un empleado, incluyendo su nombre, salario, fecha de ingreso, y calcular aumentos o descuentos.

9. Sistema de Pedidos de Comida: Desarrollar un sistema para un restaurante con clases como Plato, Pedido y Cliente. Un objeto Pedido puede contener una lista de platos que el cliente ha ordenado, y el objeto Cliente puede tener un historial de pedidos. Implementar métodos para agregar platos al pedido, calcular el total y generar la factura.
10. Venta de Entradas para Eventos: Simular un sistema de venta de entradas para eventos. Crear clases como Evento, Entrada y Venta. La clase Evento gestiona la información del evento y las entradas disponibles. La clase Venta registra la venta de entradas a los clientes.
11. Alquiler de Autos: Crear un sistema de alquiler de autos con clases como Auto, Cliente y Alquiler. La clase Auto debe tener atributos como modelo, marca y disponibilidad. La clase Alquiler debe gestionar el proceso de alquiler de autos a los clientes.
12. Sistema de reservas de transporte: Crea una jerarquía de clases para modelar un sistema de reservas de transporte.
 - a. Clase base: "Transporte" (atributos: número de identificación, origen, destino).
 - b. Clases derivadas: "Autobus", "Tren", "Avion" (atributos específicos: número de asientos, tipo de combustible, número de alas).
 - c. Implementa métodos para calcular el precio del viaje (que puede variar según el tipo de transporte) y mostrar la información de la reserva.
13. Sistema de cursos en línea: Desarrolla un sistema para gestionar cursos en línea.
 - a. Clase base: "Curso" (atributos: título, descripción, duración).
 - b. Clases derivadas: "CursoProgramacion", "CursoDiseno", "CursoMarketing" (atributos como lenguaje de programación, herramienta de diseño, tipo de marketing).
 - c. Implementa métodos para calcular el precio del curso y mostrar el temario.
14. Sistema de reproducción de medios: Crea una jerarquía de clases para representar diferentes tipos de archivos multimedia.
 - a. Clase base: "ArchivoMedia" (atributos: título, duración). ⁵
 - b. Clases derivadas: "ArchivoAudio", "ArchivoVideo", "ArchivoImagen" (atributos como formato, resolución).
 - c. Implementa un método "reproducir()" que simule la reproducción del archivo.
15. Sistema de exportación de datos: Desarrolla un sistema para exportar datos a diferentes formatos.
 - a. Clase base: "Exportador" (atributos: datos).

- b. Clases derivadas: "ExportadorCSV", "ExportadorJSON", "ExportadorXML".
 - c. Implementa un método "exportar()" que genere el archivo en el formato correspondiente.
16. Receta - Ingrediente - Cocinero:
- a. Crea una clase "Ingrediente" con atributos como nombre, cantidad y unidad de medida.
 - b. Crea una clase "Receta" que contenga una lista de ingredientes necesarios y las instrucciones de preparación.
 - c. Crea una clase "Cocinero" que pueda crear y consultar recetas.
 - d. Implementa métodos para agregar ingredientes a una receta, mostrar los ingredientes de una receta, crear una nueva receta, y buscar recetas por ingrediente.
17. Concierto - Artista - Recinto:
- a. Crea una clase "Artista" con atributos como nombre, género musical y lista de conciertos.
 - b. Crea una clase "Concierto" con atributos como fecha, hora, lugar (Recinto) y lista de artistas que participan.
 - c. Crea una clase "Recinto" con atributos como nombre, capacidad y ubicación, y que pueda tener una lista de conciertos.
 - d. Implementa métodos para agregar artistas a un concierto, mostrar los conciertos de un artista, vender entradas (gestionando la capacidad del recinto) y mostrar la lista de conciertos en un recinto.
18. Foro - Usuario - Publicación:
- a. Crea una clase "Usuario" con atributos como nombre de usuario, email y lista de publicaciones.
 - b. Crea una clase "Publicacion" (o "Post") con atributos como título, contenido, fecha y autor, y que pueda tener una lista de comentarios.
 - c. Crea una clase "Foro" que gestione una lista de usuarios y publicaciones.
 - d. Implementa métodos para crear nuevas publicaciones, comentar en publicaciones, buscar publicaciones por autor o título, y mostrar las publicaciones de un usuario.
 - e. Define como privado el contenido de la publicación y los comentarios, y crea los métodos para acceder a ellos de forma controlada.
19. Diseñar un sistema de compras que pueda manejar distintos tipos de pagos, La clase base se llamará MétodoPago y su atributo principal es el monto. Las clases derivadas:

- a. "TarjetaCrédito" (atributos: número de tarjeta, fecha de vencimiento, etc.)
- b. "TarjetaDébito" (atributo: número de tarjeta, banco, etc)
- c. "PayPal" (atributo: email,etc)
- d. "TransferenciaBancaria" (atributos: número de cuenta, banco)

Implementar usando el polimorfismo:

- e. "procesar_pago()" (realiza el pago utilizando el método específico)
 - f. "generar_recibo()" (genera un recibo con la información relevante del pago)
20. Se propone el desarrollo de un sistema para la gestión de evaluaciones en un contexto académico. El sistema modela el concepto de "evaluación" como una entidad genérica con atributos básicos como fecha de realización y valor máximo y mínimo y nota de aprobación y promoción. Se evaluarán: Examen, Tarea y Proyecto cada uno con sus propios atributos. Se requiere métodos para calcular la calificación en cada una de estas instancias.